Exp 20: EXPERIMENT TO UNDERSTAND K-MEANS CLUSTERING ALGORITHM FOR A GIVEN DATA SET.

AIM: Understand the K-means clustering algorithm for the dataset given.

SAMPLE CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans

# Load the dataset
df = pd.read_csv('Mall_Customers.csv')

# Display dataset information
df.info()

# Display first five rows
print(df.head())

# Pair plot
sns.pairplot(df)
plt.show()
```

```
# Select Annual Income and Spending Score
features = df.iloc[:, [3, 4]].values

# Apply K-Means clustering with 5 clusters
model = KMeans(n_clusters=5, random_state=0, n_init=10)
model.fit(features)

print(model)

# Create a new dataframe with features
Final = df.iloc[:, [3, 4]].copy()

# Add cluster labels
Final['label'] = model.predict(features)

# Display first five rows
print(Final.head())

# Visualize the 5 clusters
sns.set_style("whitegrid")

sns.FacetGrid(Final, hue="label", height=8) \
   .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
   .add_legend()

plt.show()
```

```
# Elbow method using Age, Annual Income and Spending Score
features_el = df.iloc[:, [2, 3, 4]].values

wcss = []

for i in range(1, 10):
    model = KMeans(n_clusters=i, random_state=0, n_init=10)
    model.fit(features_el)
    wcss.append(model.inertia_)

# Plot the elbow curve
plt.plot(range(1, 10), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

OUTPUT:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
df=pd.read_csv('Mall_Customers.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

```
C:\Users\Ayyadurai\AppData\Local\anaconda3\Lib\site-packages\sklearn\clust
er\_kmeans.py:870: FutureWarning: The default value of `n_init` will chang
e from 10 to 'auto' in 1.4. Set the value of `n_init` explicitly to suppre
ss the warning
  warnings.warn(
C:\Users\Ayyadurai\AppData\Local\anaconda3\Lib\site-packages\sklearn\clust
er\_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads. You c
an avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

```
KMeans(n_clusters=5)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

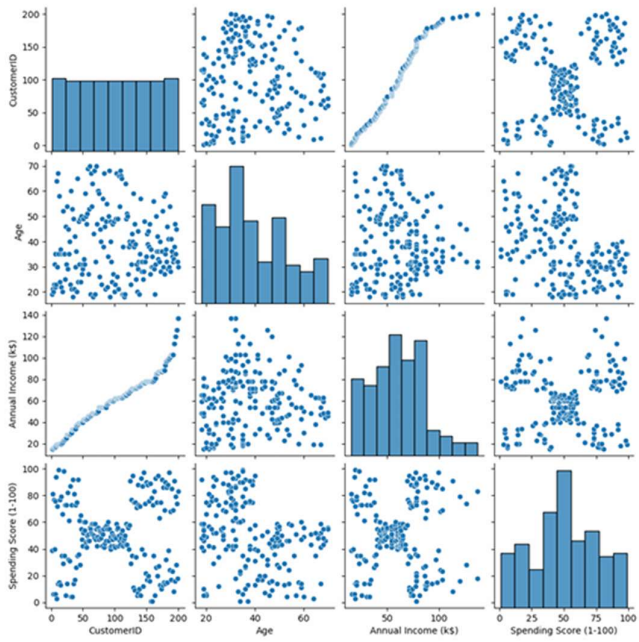
```
C:\Users\Ayyadurai\AppData\Local\Temp\ipykernel_8116\470183701.py:2: Setti
ngWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://
pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-
view-versus-a-copy)
  Final['label']=model.predict(features)
```

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4

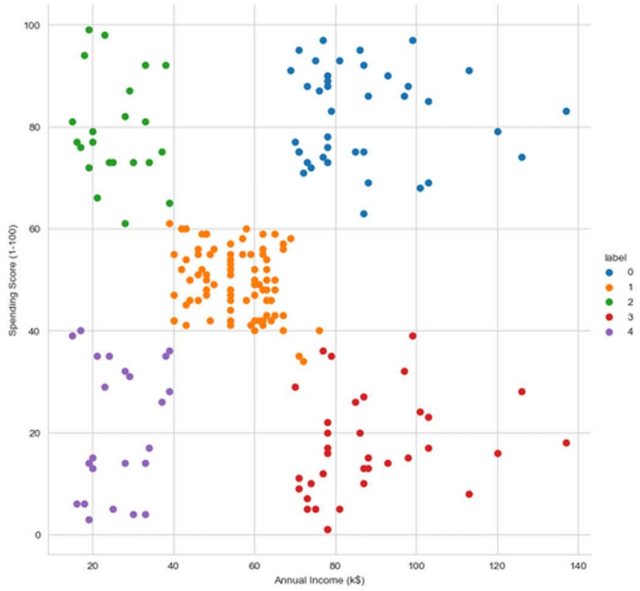
```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x170e8e47850>
```




```
features=df.iloc[:,[3,4]].values
```

```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```




```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
  model=KMeans(n_clusters=i)
  model.fit(features_el)
  wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

RESULT: Thus, experiment to understand K-means clustering algorithm for a given data set.